

THP-Mixture: Generative, Uncertainty-Aware Predictive Process Monitoring with Temporal Point Processes

Blinded for peer review

No Institute Given

Abstract. Service-based systems are increasingly monitored at runtime to keep their execution within agreed quality-of-service (QoS) and service-level-agreement (SLA) bounds. A key enabler for this is predictive process monitoring (PPM), which forecasts how ongoing executions will unfold so that operators can adapt proactively—for instance, before an SLA is violated. For such decisions, however, a prediction is only as useful as the confidence attached to it, yet today’s deep-learning PPM models are almost exclusively discriminative and deterministic: they return sharp point predictions but say little about their uncertainty. We revisit PPM through the lens of temporal point processes, a generative paradigm that natively yields predictive *distributions* over future activities and their timing, and is thus well suited to uncertainty-aware runtime monitoring. Adapting this paradigm to real-world event logs, which regularly contain events with equal timestamps, we propose *THP-Mixture*, a generative model for multi-task PPM. Across ten logs, THP-Mixture is competitive with the state of the art on conventional accuracy, provides the best-calibrated remaining time predictions, and is the fastest at inference—making it an attractive basis for online, uncertainty-aware process and service monitoring.

Keywords: Predictive process monitoring · Runtime QoS/SLA monitoring · Temporal point processes · Uncertainty quantification

1 Introduction

Modern organizations frequently realize their processes as orchestrations of software and human services, whose execution is recorded as event data [23]. Anticipating how an ongoing *case*—a single, running execution of such a process—will unfold turns passive monitoring into proactive control: it lets operators reallocate resources, escalate at-risk cases, and intervene before a case’s *quality of service* (QoS) degrades or its *service-level agreement* (SLA) is violated [11, 15]. *Predictive process monitoring* (PPM) addresses this need, forecasting the future of running cases from historical event logs [18]. Its two central time-oriented tasks are *next event prediction*, forecasting the type and time of the next event, and *remaining time prediction*, forecasting the time until a case completes.

For acting on such a forecast at runtime, however, a prediction is only as valuable as the confidence attached to it. The dominant paradigm in PPM offers no such confidence: state-of-the-art deep-learning models—whether sequence models such as LSTMs and Transformers [3, 18], or graph neural networks that encode control flow [1, 25]—are almost exclusively *discriminative* and *deterministic*, optimized directly for point predictions on downstream tasks. They return a single activity or timepoint but say little about how (un)certain it is, even though this uncertainty is exactly what runtime QoS and SLA management depend on: a case that will *probably* miss its deadline warrants a different intervention than one that almost certainly will. Underlying this limitation is that such models treat time as an auxiliary feature rather than a first-class prediction target.

Temporal point processes (TPPs) offer a principled way to close this gap. As probabilistic *generative* models of event sequences that unfold in continuous time [4, 20], they natively produce a predictive *distribution* over both the next activity and its timing—and, by extension, over a case’s remaining duration. This makes them a natural fit for uncertainty-aware service and process monitoring. Their use in PPM, however, has barely been explored, for a concrete reason: most TPPs, including the widely used class of Hawkes processes [8], assume a *simple* point process in which no two events may occur at the same moment. Real event logs frequently violate this assumption, as they routinely record consecutive events with *equal timestamps* (up to 46% in the logs we study), owing to limited timestamp resolution [6], batching [13], or parallel execution [21]. Standard TPPs are misspecified for such logs: their predictive distributions place zero probability on the very transitions the data contains, forfeiting the distributional benefits that motivate their use.

Therefore, we propose *THP-Mixture*, an adaptation of the Transformer Hawkes Process (THP) [28] that brings the generative TPP paradigm to multi-task PPM. Inspired by the intensity-free formulation [19], we replace THP’s conditional intensity and its time prediction head with a *mixture time head* that can place probability on equal timestamps. We train it jointly with a next activity head, so that the model predicts the next activity and its time together. From the resulting generative model, we then obtain an uncertainty-aware remaining time distribution through Monte Carlo rollout. THP-Mixture is thus the first generative temporal point process for multi-task PPM that natively handles the equal timestamps that are pervasive in real event logs.

We evaluate *THP-Mixture* on ten real-world event logs against representative state-of-the-art PPM methods, focusing not on point accuracy alone but also on what matters for runtime monitoring—calibrated uncertainty and low-latency inference—and observe that:

- under conventional point metrics, *THP-Mixture* is competitive, though not dominant: it attains the lowest next event time error on high-frequency, small-vocabulary logs, whereas state-of-the-art sequence models retain an edge in next activity accuracy on larger, lower-frequency logs;
- evaluated as a probabilistic predictor, *THP-Mixture* yields the best-calibrated remaining time predictions among all evaluated methods—the property most directly relevant to proactive SLA and QoS management; and

- *THP-Mixture* attains the lowest overall training cost and the fastest inference on all ten logs, scoring each prefix’s next event prediction in a single non-autoregressive forward pass, which makes it attractive for online, low-latency runtime monitoring.

In the remainder, Section 2 introduces preliminaries and Section 3 motivates our work. Section 4 presents the *THP-Mixture* model, Section 5 reports our evaluation, Section 6 discusses related work, and Section 7 concludes.

2 Preliminaries

This section defines the notation and prediction tasks used throughout the paper.

Event data. A trace $\sigma = \langle e_1, \dots, e_{|\sigma|} \rangle$ is a sequence of events of a case, and an event log L is a collection of traces of the same process. Each event e_i is a tuple (a_i, c, t_i) with activity a_i , case id c , and timestamp t_i ; we write $\tau_i = t_i - t_{i-1}$ (for $i > 1$, with $\tau_1 = 0$) for the inter-event time. A *prefix* $\mathcal{H}_k(\sigma) = \langle e_1, \dots, e_k \rangle$, $1 \leq k < |\sigma|$, captures the execution of a case up to its k -th event.

Prediction tasks. Given a prefix $\mathcal{H}_k(\sigma)$, *next event prediction* forecasts the next activity and the next event timestamp, whereas *remaining time prediction* forecasts the time until the case completes. We capture these with three predictors that each map a prefix to a prediction: $\hat{a}$ for the next activity a_{k+1} , $\hat{t}$ for the next time t_{k+1} , and $\hat{r}$ for the remaining time until completion. Formally,

$$\hat{a}(\mathcal{H}_k(\sigma)) \approx a_{k+1}, \quad \hat{t}(\mathcal{H}_k(\sigma)) \approx t_k + \tau_{k+1}, \quad \hat{r}(\mathcal{H}_k(\sigma)) \approx t_{|\sigma|} - t_k.$$

3 Motivation

This section introduces temporal point processes (TPPs), explains why they suit PPM, and shows how the limitations of a state-of-the-art TPP model motivate our work.

Temporal point processes. TPPs provide a probabilistic generative framework for sequences of discrete events in continuous time [4]. Conditioned on the history of past events, a TPP defines a joint distribution over when the next event occurs and which type it carries¹, typically parameterized through a conditional intensity function λ^* or a conditional density p^* . Classical models such as the Hawkes process [8] use handcrafted parametric intensities, whereas recent neural TPPs replace these with deep neural networks [5, 14, 28].

Suitability for PPM. Three properties make TPPs a natural fit for PPM. First, the modeled object matches: a TPP models event times in the continuous domain $\mathbb{R}^+$, so the traces in an event log are realizations of an underlying TPP and an activity’s occurrence time becomes a prediction target in its own right. Second, being generative, a TPP yields a full predictive distribution rather than a point prediction, providing the calibrated uncertainty that runtime QoS and

¹ A TPP whose events carry types is formally a marked TPP with a finite mark space [4]; following THP, we refer to marks as event types [28].

SLA management require [7, 20]. Third, TPPs are multi-task by construction: a single one-step conditional model supports next activity and next-time prediction directly, and yields remaining time prediction by sampling continuations of the running case, without task-specific decoders.

Transformer Hawkes Process. Despite this fit, neural TPPs remain largely unexplored for predictive monitoring, with only a first attempt in this direction [17]. A state-of-the-art neural TPP is the Transformer Hawkes Process (THP) [28], which takes an *event sequence* of (timestamp, event-type) pairs and, through N causally masked self-attention layers followed by an LSTM (an optional configuration provided in the official THP codebase), produces a hidden state $\mathbf{h}(t_k)$ that summarizes the prefix $\mathcal{H}_k(\sigma)$. We refer to this encoder, shown in Figure 1, as the *THP-backbone*. Its output layer, however, assigns zero probability to equal timestamps, so that any prefix containing simultaneous events receives zero likelihood—even though such timestamps are pervasive in event logs. This misspecification motivates the *THP-Mixture* model presented next.

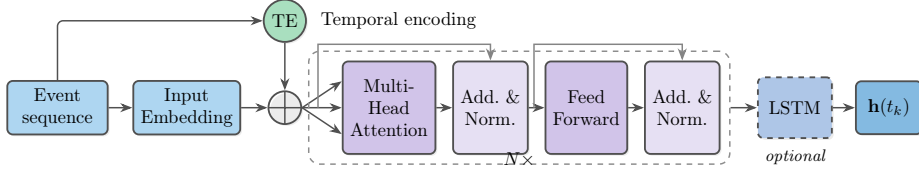


Fig. 1. The THP-backbone, mapping an event sequence to the hidden state $\mathbf{h}(t_k)$.

4 THP-Mixture

We propose *THP-Mixture*, a probabilistic generative model for multi-task PPM, with an overview shown in Figure 2. Given a prefix of a running case, THP-Mixture predicts a joint distribution over the next activity, next event time, and remaining time. Because each prediction is derived from this distribution rather than produced as a point prediction, the model is uncertainty-aware, yielding a predictive distribution over the remaining time. As its name suggests, it keeps the THP-backbone as its sequence encoder but replaces THP’s intensity-based output with a zero-inflated LogNormal mixture density, which—unlike the original—places probability mass on the equal timestamp pervasive in event logs.

Section 4.1 describes how event data is cast into the format the model consumes—prefixes drawn from completed traces at training time and a prefix of an ongoing case at inference time, while Section 4.2 presents the model itself: its architecture, training objective, and inference procedure.

4.1 Data Transformation

As motivated in Section 3, a completed trace σ can be regarded as a realization of a TPP, and a prefix $\mathcal{H}_k(\sigma)$ as a partially observed realization—the history upon which a TPP is conditioned up to timestamp t_k . This section makes that

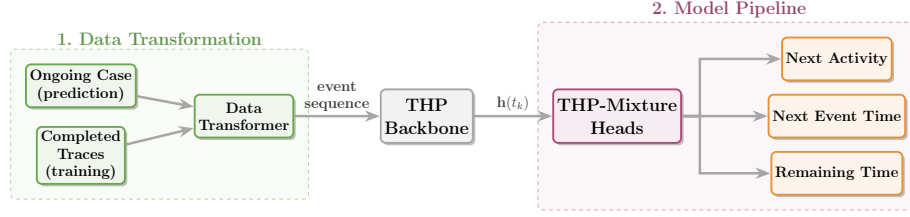


Fig. 2. Overview of THP-Mixture for multi-task predictions in PPM

correspondence operational at two levels, illustrated in Figure 2. At the *event level*, a projection casts each event into the pair the model consumes. At the *trace level*, three mismatches remain between event log traces and TPP realizations—two resolved here by restructuring the data, one deferred to the model.

Projection. Each event e_i in $\mathcal{H}_k(\sigma)$ is projected onto the pair (a_i, t_i) : the activity a_i serves as the event type, and t_i is the elapsed time since the first event of the case.

Termination. Cases terminate, whereas TPP realizations are unbounded. Since the TPP carries no explicit termination symbol, we resolve this at inference (Section 4.2). The activity set $\mathcal{A}$ is simply the activities occurring in the log.

Case separation. An event log interleaves many concurrent cases, whereas a TPP realization is a single sequence. We therefore treat each case as its own realization operating under its own clock, never concatenating cases into a global stream.

Equal timestamps. Unlike the previous two, this mismatch cannot be removed by restructuring the data. Equal timestamps violate the *simple* point-process assumption that no two events occur at the same moment; the model itself accounts for them (Section 4.2).

4.2 Model Pipeline

After transformation, we now present the model itself, following the pipeline shown in Figure 3: its architecture, which equips the THP-backbone with new prediction heads; its training objective, which fits the resulting conditional density of the next event by maximum likelihood; and its inference procedure, which decodes the PPM tasks from this density.

Architecture. We keep the THP-backbone and replace its output layer with three linear heads that jointly parameterize a conditional density over the next event, rather than emitting point predictions.

Prediction heads. Unlike THP’s single event type and time heads, we apply three linear heads to $\mathbf{h}(t_k) \in \mathbb{R}^M$:

1. a tie head $\mathbf{w}^\pi \in \mathbb{R}^M$ producing a zero-inflation logit
2. a mixture head $\mathbf{W}^m \in \mathbb{R}^{3K \times M}$ with K representing the number of LogNormal components; and

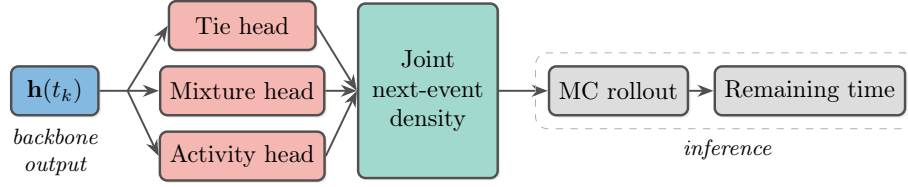


Fig. 3. Prediction heads and inference procedure of THP-Mixture.

3. an activity head $\mathbf{W}^a \in \mathbb{R}^{|\mathcal{A}| \times M}$ producing next activity logits over $\mathcal{A}$

Writing $\tau := \tau_{k+1}$, the tie head gives the logit of π , the probability of an instantaneous transition ($\tau = 0$); the mixture head parameterizes the density of τ given $\mathbf{h}(t_k)$, with K components capturing the multimodal inter-event times of event logs (e.g. fast automated handoffs alongside slow human steps) that a single LogNormal cannot. Together the three heads define the conditional joint density over the next activity and its time (Equation 1), learned by maximum likelihood.

Training. We model the conditional joint density of the next activity and time given the prefix $\mathcal{H}_k(\sigma)$, using the asterisk (*) to denote this conditioning by convention:

$$p^*(a, \tau) = p^*(a) p^*(\tau). \quad (1)$$

This factorization assumes activity and time are conditionally independent given the prefix, a standard simplification in neural TPPs [20]; their dependence is captured only through the shared representation $\mathbf{h}(t_k)$.

Activity density. The next activity follows a categorical distribution over $\mathcal{A}$,

$$p^*(a) = \text{softmax}(\mathbf{W}^a \mathbf{h}(t_k))_a, \quad (2)$$

where the subscript a extracts the entry corresponding to activity a from the resulting probability vector.

Zero-inflated LogNormal mixture. We model τ with a zero-inflated mixture: with probability π the next event is instantaneous ($\tau = 0$), and with probability $1 - \pi$, the time is drawn from a K -component LogNormal mixture,

$$p^*(\tau) = \pi \delta_0(\tau) + (1 - \pi) \sum_{m=1}^K w_m \text{LN}(\tau; \mu_m, s_m), \quad (3)$$

where δ_0 denotes the point mass at 0, m indexes the mixture components with weights $w_m \geq 0$, $\sum_{m=1}^K w_m = 1$, and $\text{LN}(\tau; \mu_m, s_m) = \frac{1}{\tau s_m \sqrt{2\pi}} \exp\left(-\frac{(\ln \tau - \mu_m)^2}{2s_m^2}\right)$ is the LogNormal density with μ_m and s_m the mean and standard deviation of $\ln \tau$ under component m . The parameters are derived from the heads at position k : the zero-inflation probability $\pi = \text{sigmoid}(\mathbf{w}^\pi \mathbf{h}(t_k))$, the mixture weights $\mathbf{w} = \text{softmax}(\cdot)$, the log-scale means $\boldsymbol{\mu}$, and the positive scales $\mathbf{s} = \text{softplus}(\cdot) + \varepsilon$, where $(\mathbf{w}, \boldsymbol{\mu}, \mathbf{s})$ jointly constitute the $3K$ outputs of $\mathbf{W}^m \mathbf{h}(t_k)$.

Maximum likelihood estimation. Each prediction position contributes a time term and an activity term. The negative log-likelihood of the inter-event time is

$$\ell_\tau = \begin{cases} -\log \pi, & \tau = 0, \\ -\log(1 - \pi) - \log \sum_{m=1}^K w_m \text{LN}(\tau; \mu_m, s_m), & \tau > 0, \end{cases} \quad (4)$$

and the activity term is defined as the cross-entropy loss $\ell_a = -\log p^*(a_{k+1})$. The training objective sums both terms over all prediction positions of the N training cases,

$$\min_{\theta} \sum_{i=1}^N \sum_{k=1}^{|\sigma_i|-1} (\ell_\tau^{(i,k)} + \ell_a^{(i,k)}), \quad (5)$$

where θ encompasses all learnable parameters of the backbone and the three heads.

Inference. Given a prefix $\mathcal{H}_k(\sigma)$ (an ongoing case), the next activity predictor is the mode of the categorical distribution,

$$\hat{a}(\mathcal{H}_k(\sigma)) = \arg \max_{a \in \mathcal{A}} p^*(a), \quad (6)$$

and the next event time predictor is the closed-form expectation of Equation 3,

$$\hat{t}(\mathcal{H}_k(\sigma)) = \mathbb{E}[\tau \mid \mathcal{H}_k(\sigma)] = (1 - \pi) \sum_{m=1}^K w_m \exp\left(\mu_m + \frac{1}{2} s_m^2\right), \quad (7)$$

recovering the next timestamp as $t_k + \hat{t}(\mathcal{H}_k(\sigma))$. The remaining time predictor accumulates these one-step expectations across the suffix, conditioning on the true prefix at each step:

$$\hat{r}(\mathcal{H}_k(\sigma)) = \sum_{j=k}^{|\sigma|-1} \mathbb{E}[\tau \mid \mathcal{H}_j(\sigma)], \quad (8)$$

where the ground truth supplies both the horizon $J = |\sigma| - k$ and the intermediate events, since $\mathcal{A}$ lacks an end-of-case symbol. Consequently, Equation 8 is a teacher-forced plug-in predictor rather than the model’s true generative expectation. For an uncertainty-aware prediction, we drop this conditioning: we ancestrally sample S continuations over the same horizon J , and their accumulated times form an empirical remaining time distribution summarized by its sample mean and standard deviation.

5 Evaluation

We evaluate THP-Mixture along three dimensions: point predictive accuracy, uncertainty quality, and computational efficiency. We first describe the datasets, baselines, metrics, and experimental protocol, and then discuss the obtained results. Our implementation and additional results are available in the public repository of our project ².

² <https://anonymous.4open.science/r/thp-mixture-56CA/>

5.1 Experimental Setup

Datasets. We use ten public event logs spanning clinical, incident and request management, financial, and travel-reimbursement processes; Table 1 reports their statistics after preprocessing. To relate model behavior to log characteristics, we cluster the logs with Ward’s method [24] on the two statistics that drive TPP modeling difficulty, the median inter-event time and the number of distinct activities (shaded in Table 1). This yields a high-frequency, small-vocabulary group (Group A), a low-frequency, larger-vocabulary group (Group B), and one very-high-frequency, large-vocabulary outlier (BPIC15-1).

Table 1. DATASET STATISTICS. Cases, Events and Act: numbers of cases, events, and distinct activities; Avg. len: mean case length; Med. Δt : median inter-event time; Avg. dur (d): mean case duration in days. Shaded columns (Act, Med. Δt) are the cluster-defining statistics.

Log	Cases	Events	Act	Avg. len	Med. Δt	Avg. dur (d)
Group A — high-frequency, small-vocab						
SEPSIS	1 049	15 214	16	14.5	6.5 min	28.5
BPIC13I	7 554	65 533	4	8.7	10.1 min	12.1
BPIC12	13 087	262 200	24	20.0	48 s	8.6
Group B — low-frequency, relatively large-vocab						
HELPDESK	4 580	21 348	14	4.7	3.28 d	40.9
BPIC20DD	10 500	56 437	17	5.4	1.02 d	11.5
BPIC20RFP	6 886	36 796	19	5.3	1.11 d	12.0
BPIC20PTC	2 099	18 246	29	8.7	22.5 h	36.8
BPIC20ID	6 449	72 151	34	11.2	2.00 d	86.5
BPIC20TPD	7 065	86 581	51	12.3	1.74 d	87.4
Outlier — very-high-frequency, large-vocab						
BPIC15-1	1 199	52 217	398	43.6	1 s	95.7

Preprocessing. All event logs are preprocessed following the strict temporal-splitting approach [26], via the following steps:

Duplicate removal and case-duration trimming. We drop all fully duplicated events (712 across the ten datasets). Rather than specifying a fixed maximum case duration as in the original approach, we trim the longest 5% of cases, yielding a data-dependent threshold that adapts consistently across all datasets.

End-of-dataset debiasing. We drop cases that start too late to plausibly complete within the observed window, i.e., those whose start time exceeds the final log timestamp minus the longest retained case duration (3,274 cases).³

³ For BPIC13I, case arrivals concentrate near the end of the log, so this step would leave too few training cases; it is skipped for this dataset.

Table 2. EVALUATION METRICS FOR DOWNSTREAM TASKS. $\uparrow/\downarrow$ mark whether higher/lower is better.

Metric	Task	Description	Range
Point metrics			
Acc	next activity	Fraction of correct next activities	$[0, 1]$ $\uparrow$
MAE	next event & remaining time	Mean absolute error of the predicted time	$\mathbb{R}^+$ $\downarrow$
Uncertainty metrics			
MA	remaining time	Calibration: gap between empirical and nominal coverage	$[0, 0.5]$ $\downarrow$
MPIW	remaining time	Sharpness: mean prediction-interval width	$\mathbb{R}^+$ $\downarrow$
AURG	remaining time	Sparsification: error drop when rejecting uncertain cases	$\mathbb{R}$ $\uparrow$

Strict temporal split. The most recent 20% of cases by start time form the test set; the training set retains only those cases that complete before the earliest test-case start time.

Compared methods. We compare THP-Mixture with models from four families. *Neural TPP (THP-B, THP-M).* Two variants of THP [28] sharing the same backbone and next activity head: *THP-B* (Baseline) with the original point-regression time head, which enters only the point-prediction comparisons; and our proposed *THP-M* (Mixture) with the zero-inflated LogNormal time head for probabilistic prediction.

State-of-the-art multi-task PPM models (SuTraN, ED-LSTM). *SuTraN* [27], a Transformer encoder-decoder, and *ED-LSTM* [10, 22, 27], an encoder-decoder LSTM, both designed for multi-task prediction.

Classical TPP (H-uni, H-mk). Two variants of an exponential-kernel Hawkes process [8] fitted by maximum likelihood: a univariate model on inter-event times only (*H-uni*), and a marked model with one intensity dimension per activity (*H-mk*).

Uncertainty-aware models (LA-CR $\star$, UQ $\star$). *LA-CR $\star$* represents the per-metric best performance across the two published calibration variants (LA+I, LA+S) of LA-CR [2], retrained under our strict temporal split. *UQ $\star$* applies the same composite convention to the values reported in [2] under their original dataset split, i.e., the per-metric best across the eight UQ methods they evaluated.

Evaluation metrics. We report two families of metrics (Table 2): point metrics for all three tasks, and uncertainty metrics that score the remaining time predictive distribution. The latter are identical to those used by Amiri et al. [2].

Execution settings. All neural models are trained with early stopping (patience 24), except for LA-CR, which follows its original protocol [2]. THP converged within at most 101 epochs, compared to 171 for SuTraN and 169 for ED-LSTM;

it uses a learning rate of 2×10^{-3} and gradient clipping at norm 1.0 to stabilize the mixture time head. H-uni and H-mk are fitted by maximum likelihood, with equal timestamps in inter-events perturbed by less than one second to avoid numerical instability. All neural models are trained on a single NVIDIA RTX A6000 GPU, while the Hawkes processes are estimated on a 32-vCPU Intel Xeon CPU. The Monte Carlo rollout uses $S = 50$ sampled continuations per prefix; as THP defines no terminal event, the rollout horizon is set to the true number of remaining events for each test prefix. All reported figures are test set results.

5.2 Results and Discussion

Next event predictions. Table 3 compares the models on next activity and next event time prediction using the point metrics described in Section 5.1. H-uni, which models inter-event times without marks, yields no next activity prediction (“-”).

For next activity prediction, SuTraN and ED-LSTM achieve the highest accuracy on 8 of the 10 datasets. However, THP-M is highly competitive in Group A: it attains the best overall accuracy on BPIC12 (87.3% vs. 86.1% for THP-B and 84.5% for ED-LSTM) and the highest accuracy among neural models on BPIC13I (67.0% vs. 50.2% for ED-LSTM, the best among the state-of-the-art multi-task PPM models). Overall, THP-B and THP-M exhibit similar next activity accuracy, though notable gaps remain on certain Group B datasets (e.g., 80.3% vs. 71.1% on BPIC20DD).

Next event time prediction is more clearly group-dependent. THP-B and THP-M perform particularly well on high-frequency, small-vocabulary datasets (Group A) — on SEPSIS, THP-M reaches an MAE of 0.26 d, more than $4\times$ lower than that of SuTraN (1.15 d) and ED-LSTM (1.10 d) — whereas SuTraN and ED-LSTM tend to achieve lower time errors on low-frequency datasets with more complex activity vocabularies (Group B). Overall, no model consistently dominates the conventional point metrics across all datasets.

Remaining time prediction. Table 4 reports remaining time prediction results. Under the conventional point metric (MAE), there is no clear winner: some models achieve the lowest MAE on a similar number of datasets (e.g., 4/10 for THP-M, 3/10 for ED-LSTM). As in the next event prediction task, THP-M tends to perform better in Group A (e.g., on SEPSIS it attains 9.46 d against 14.83 d for SuTraN).

Under uncertainty metrics, THP-M achieves the lowest miscalibration area (MA) on 6 of the 10 datasets, winning on more datasets than either UQ $\star$ or LA-CR $\star$, and the highest AURG on 4 datasets. Its main weakness is sharpness: THP-M does not achieve the lowest MPIW on any dataset, indicating that the competing recalibrated regression methods generally produce narrower prediction intervals.

Horizon asymmetry. The remaining time evaluation is asymmetric. THP-M relies on the oracle remaining length J (Section 4.2) for Monte Carlo rollouts and the ground-truth suffix for point predicts, inherently teacher-forcing intermediate

Table 3. NEXT EVENT PREDICTIONS.

Log	Next activity accuracy (%) $\uparrow$						Next event time MAE (days) $\downarrow$					
	THP-B	THP-M	SuTraN	ED-LSTM	H-uni	H-mk	THP-B	THP-M	SuTraN	ED-LSTM	H-uni	H-mk
Group A												
SEPSIS	62.3	61.8	59.9	62.5	–	50.6	0.27	0.26	1.15	1.10	0.46	0.46
BPIC13I	61.9	67.0	48.3	50.2	–	67.9	0.75	0.77	0.92	0.87	0.87	0.86
BPIC12	86.1	87.3	77.3	84.5	–	67.0	0.30	0.29	0.41	0.34	0.56	0.58
Group B												
HELPDESK	56.0	60.3	73.8	73.8	–	44.4	6.66	6.83	4.92	4.61	7.57	7.26
BPIC20DD	80.3	71.1	91.1	88.9	–	59.8	1.38	1.40	1.31	1.35	1.86	1.68
BPIC20RFP	82.0	80.8	89.8	87.2	–	57.2	1.61	1.60	1.55	1.55	2.10	1.88
BPIC20PTC	78.2	77.5	80.9	87.4	–	48.3	3.64	3.84	2.13	2.04	5.08	4.64
BPIC20ID	80.1	81.2	90.3	89.7	–	48.8	6.84	6.54	3.27	3.32	9.81	9.12
BPIC20TPD	76.0	69.7	84.0	84.1	–	42.3	4.91	5.15	4.86	4.83	7.10	6.64
Outlier												
BPIC15-1	6.6	8.9	16.5	10.6	–	0.3	2.16	2.21	1.53	1.60	2.21	1.81

events. In contrast, other models predict the remaining time via a dedicated head without access to this true event count. This structural advantage affects THP-M’s entire remaining time block, encompassing both the point MAE and uncertainty metrics. Consequently, these results should be interpreted with this caveat. Restoring horizon parity via a self-terminating end-of-case symbol is left to future work.

Efficiency. Table 5 shows that the distributional gains cost nothing at deployment. The Total/Mean row sums training cost and averages inference cost over the 10 logs; the final row reports mean training time per epoch. LA-CR carries no $\star$ here since both calibration variants share the same trained network and Laplace fit, incurring no additional training cost; as the fit is not epoch-based, its per-epoch entry is left as “–”. The Hawkes column reports one timed run covering both the H-uni and H-mk models. Among the neural models, THP-M is the cheapest to train (208 min vs. 706 min and 316 min for SuTraN and ED-LSTM, respectively, and 1541 min for LA-CR) and the fastest at inference: first or second on all 10 datasets, averaging 0.030 ms per prefix ($52\times$ faster than SuTraN, $20\times$ than ED-LSTM, and $4.6\times$ than LA-CR). Although ED-LSTM is the cheapest per epoch, THP converges in fewer epochs (Section 5.1) and thus wins on total cost. Only the classical Hawkes models are cheaper to train in total, at far lower accuracy; their closed-form prediction runs on CPU and therefore is not benchmarked for inference. The inference figures measure each model’s standard test pass per

Table 4. REMAINING TIME PREDICTION: the point metric RT-MAE, and the uncertainty metrics MA, MPIW, and AURG among the uncertainty-aware models (THP-M, LA-CR*, UQ*); best per block in bold, THP-Mixture columns shaded.

Log	RT-MAE (d) ↓					MA ↓			MPIW ↓			AURG ↑		
	THP-B	THP-M	SuTraN	ED-LSTM	UQ*	THP-M	LA-CR*	UQ*	THP-M	LA-CR*	UQ*	THP-M	LA-CR*	UQ*
Group A														
SEPSIS	27.42	9.46	14.83	15.08	15.32	0.04	0.08	0.04	14.05	3.55	4.86	6.63	4.17	8.35
BPIC13I	8.57	5.55	3.20	3.47	3.06	0.04	0.12	0.04	7.00	0.37	8.05	1.99	0.20	1.43
BPIC12	6.76	3.48	4.69	7.85	5.86	0.04	0.06	0.05	73.85	2.76	8.63	1.80	1.26	1.86
Group B														
HELPDESK	8.82	10.86	4.22	3.77	9.45	0.05	0.05	0.12	26.80	16.50	7.63	2.83	1.35	1.48
BPIC20DD	3.56	3.21	3.25	3.61	4.12	0.08	0.02	0.04	8.16	3.59	3.37	1.01	0.47	2.05
BPIC20RFP	4.89	3.84	3.97	4.33	5.09	0.03	0.03	0.03	38.67	2.80	5.29	1.14	0.47	2.61
BPIC20PTC	12.03	12.65	6.18	6.11	7.73	0.04	0.05	0.05	65.07	6.94	5.13	6.59	5.04	5.71
BPIC20ID	22.05	22.13	10.13	10.10	14.08	0.03	0.10	0.03	60.13	2.94	15.77	11.29	11.45	8.36
BPIC20TPD	25.61	22.56	17.60	18.26	25.02	0.06	0.06	0.05	34.89	10.63	35.57	10.48	4.41	14.45
Outlier														
BPIC15-1	51.42	30.49	28.32	31.39	24.71	0.03	0.11	0.03	3276.69 [‡]	16.85	20.25	16.56	7.01	12.61
THP-M best		4/10				6/10			0/10			4/10		

prefix: a single non-autoregressive forward pass for THP, and a full autoregressive suffix decode for SuTraN and ED-LSTM (their standard implemented inference mode). For THP-M, the remaining time distribution additionally requires the Monte Carlo rollout of Section 4.2, whose cost grows linearly with S and the rollout length, and is excluded from Table 5.

Overall, the results reveal a trade-off rather than uniform dominance. THP-M is competitive on conventional point metrics, particularly for high-frequency logs, while its clearest advantages lie in remaining time calibration and inference efficiency. At the same time, discriminative sequence models remain stronger on next activity prediction for several larger-vocabulary logs, and recalibrated regression methods produce sharper remaining time intervals.

6 Related Work

PPM can be seen as a form of *predictive service monitoring* in service-oriented computing: it forecasts the behavior of running services or process executions to enable proactive adaptation, such as anticipating SLA or deadline violations before they occur [11, 15]. Whereas these works predict service performance with discriminative and deterministic models, we approach PPM with a generative and probabilistic model: a temporal point process. We relate our work to three research directions in PPM: sequence models, GNNs, and TPPs for event prediction.

Table 5. TRAINING AND INFERENCE COST OF ALL MODELS (bold: fastest; blue: second fastest).

Log	Training (min) ↓						Inference (ms/prefix) ↓				
	THP-B	THP-M	SuTraN	ED-LSTM	Hawkes	LA-CR	THP-B	THP-M	SuTraN	ED-LSTM	LA-CR
Group A											
SEPSIS	17.62	5.93	27.08	3.28	0.97	53.40	0.02	0.02	1.40	0.63	0.32
BPIC13I	15.65	4.75	41.92	25.53	0.50	5.90	0.02	0.02	0.92	0.37	0.15
BPIC12	401.24	82.70	239.40	179.93	15.89	1090.80	0.01	0.01	4.99	1.65	0.23
Group B											
HELPDESK	14.70	15.39	8.22	2.27	1.32	8.10	0.07	0.07	0.25	0.13	0.06
BPIC20DD	18.02	14.85	44.27	5.72	2.28	32.70	0.05	0.05	0.17	0.09	0.07
BPIC20RFP	16.03	13.19	22.55	2.83	1.73	17.80	0.06	0.05	0.22	0.11	0.07
BPIC20PTC	5.07	7.49	9.10	3.05	0.78	15.70	0.04	0.04	0.34	0.17	0.06
BPIC20ID	15.31	14.41	31.45	5.25	2.12	38.80	0.02	0.02	0.37	0.17	0.07
BPIC20TPD	27.00	26.01	84.74	15.89	3.73	125.30	0.02	0.02	0.70	0.32	0.18
Outlier											
BPIC15-1	38.56	23.73	197.35	72.92	3.84	153.40	0.01	0.01	6.28	2.41	0.18
Total / Mean	569.20	208.45	706.08	316.67	33.15	1541.90	0.03	0.03	1.56	0.60	0.14
Mean (min/epoch)	0.64	0.59	0.89	0.26	3.32	—					

Sequence models for PPM. A comprehensive benchmark found that no single architecture dominates across logs, with RNNs generally outperforming CNNs [18]; more recent sequence models are predominantly Transformer-based [3, 27]. These models reach state-of-the-art accuracy, but are discriminative and sequence-driven: they operate on discrete steps and treat time as an auxiliary feature [12, 16] rather than modeling its continuous dynamics.

GNNs for PPM. GNN-based methods instead capture the structural, non-linear dependencies in event logs [1, 9, 25]; PGTNet [1], for instance, combines graph representations with temporal features for remaining time prediction. Recently, the focus of GNNs has shifted towards discovering more complex dependencies in event logs, rather than developing uncertainty-aware algorithms; we therefore omit GNNs from our evaluation.

TPPs for event prediction. TPPs are an established generative paradigm for event sequences in continuous time, from the classical Hawkes process [8] to neural variants [5, 14] such as the THP [28]. Their use for PPM, however, has seen only a single preliminary attempt [17]. To our knowledge, we are the first to adapt a neural TPP to multi-task PPM and to handle the equal timestamps pervasive in real service execution logs, bringing generative, uncertainty-aware prediction to predictive service monitoring.

7 Conclusion

We investigated temporal point processes (TPPs) as a generative paradigm for multi-task PPM and proposed *THP-Mixture*, which extends the Transformer Hawkes Process to natively handle the equal timestamps of real event logs while maintaining joint prediction of the next activity and its time. On ten real-world logs, THP-Mixture is competitive—though not dominant—under conventional point metrics, yet it is the best-calibrated remaining time predictor among the evaluated methods and, among the neural models, the cheapest to train and the fastest at inference. These properties are precisely what runtime service monitoring requires: a calibrated distribution lets an operator act on the *risk* of an SLA or deadline breach rather than on a bare point prediction, while single-pass inference keeps that decision within the latency budget of an online service. The value of the generative paradigm thus lies not in point accuracy, but in making predictive monitoring both trustworthy and operational.

Several directions follow. Beyond richer TPP models and additional PPM tasks such as suffix prediction, the most promising next step is to close the loop from prediction to action: feeding THP-Mixture’s predictive distribution into the runtime adaptation of service compositions, so that decisions such as SLA-preserving rerouting or admission control are driven by quantified risk rather than point forecasts. Progress here is nonetheless bounded by data—coarse timestamp granularity and the scarcity of diverse, high-quality event logs limit what any model can learn—which we hope encourages the community to evaluate PPM beyond point accuracy and to invest in richer benchmarks. Establishing TPPs as a generative, uncertainty-aware foundation for predictive service monitoring is, we believe, a promising step in this direction.

References

1. Amiri Elyasi, K., van der Aa, H., Stuckenschmidt, H.: Pgtnet: A process graph transformer network for remaining time prediction of business process instances. In: CAiSE. pp. 124–140. Springer (2024)
2. Amiri Elyasi, K., van der Aa, H., Stuckenschmidt, H.: A simple and calibrated approach for uncertainty-aware remaining time prediction. In: BPM. pp. 217–234. Springer (2025)
3. Bukhsh, Z.A., Saeed, A., Dijkman, R.M.: Processtransformer: Predictive business process monitoring with transformer network (2021)
4. Daley, D.J., Vere-Jones, D.: An introduction to the theory of point processes: volume II: general theory and structure. Springer (2008)
5. Du, N., Dai, H., Trivedi, R., Upadhyay, U., Gomez-Rodriguez, M., Song, L.: Recurrent marked temporal point processes: Embedding event history to vector. In: KDD. pp. 1555–1564 (2016)
6. Fischer, D.A., Goel, K., Andrews, R., van Dun, C.G.J., Wynn, M.T., Röglinger, M.: Enhancing event log quality: Detecting and quantifying timestamp imperfections. In: BPM. pp. 309–326. Springer (2020)
7. Ghahramani, Z.: Probabilistic machine learning and artificial intelligence. *Nature* **521**(7553), 452–459 (2015)

8. Hawkes, A.G.: Point spectra of some mutually exciting point processes. *JRSS-B* pp. 438–443 (1971)
9. Hennig, M.C., Schmidt, R., Möhring, M.: Temporal GNNs for remaining time prediction: an evaluation. In: *ICPM*. pp. 1–8. IEEE (2025)
10. Ketykó, I., Mannhardt, F., Hassani, M., Van Dongen, B.F.: What averages do not tell: predicting real life processes with sequential deep learning. In: *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*. pp. 1128–1131 (2022)
11. Leitner, P., Wetzstein, B., Rosenberg, F., Michlmayr, A., Dustdar, S., Leymann, F.: Runtime prediction of service level agreement violations for composite services. In: *ICSOC Workshops*. pp. 176–186. Springer (2010)
12. Lischka, A., Rauch, S., Stritzel, O.: Directly follows graphs go predictive process monitoring with graph neural networks. *arXiv preprint arXiv:2503.03197* (2025)
13. Martin, N., Swennen, M., Depaire, B., Jans, M., Caris, A., Vanhoof, K.: Batch processing: Definition and event log identification. In: *SIMPDA* (2015)
14. Mei, H., Eisner, J.M.: The neural hawkes process: A neurally self-modulating multivariate point process. *NeurIPS* (2017)
15. Metzger, A., Leitner, P., Ivanović, D., Schmieders, E., Franklin, R., Carro, M., Dustdar, S., Pohl, K.: Comparing and combining predictive business process monitoring techniques. *IEEE Trans. Syst. Man Cybern.: Syst.* **45**, 276–290 (2015)
16. Nguyen, A., Chatterjee, S., Weinzierl, S., Schwinn, L., Matzner, M., Eskofier, B.: Time matters: Time-aware lstms for predictive business process monitoring. In: *ICPM*. pp. 112–123. Springer (2020)
17. Nguyen, T.T.H., Tran, T.C.T., Truong, T.C.M.: Temporal point processes for business process monitoring. *Quy Nhon University Journal of Science*. doi **10** (2024)
18. Rama-Maneiro, E., Vidal, J.C., Lama, M.: Deep learning for predictive business process monitoring: Review and benchmark. *IEEE Transactions on Services Computing* **16**(1), 739–756 (2021)
19. Shchur, O., Bilos, M., Günnemann, S.: Intensity-free learning of Temporal Point Processes. In: *ICLR* (2020)
20. Shchur, O., Türkmen, A.C., Januschowski, T., Günnemann, S.: Neural temporal point processes: A review pp. 4585–4593 (2021)
21. Suriadi, S., Andrews, R., ter Hofstede, A.H., Wynn, M.T.: Event log imperfection patterns for process mining: Towards a systematic approach to cleaning event logs. *Inf. Syst. J.* **64**, 132–150 (2017)
22. Taymouri, F., La Rosa, M., Erfani, S.M.: A deep adversarial model for suffix and remaining time prediction of event sequences. In: *SDM*. pp. 522–530. SIAM (2021)
23. Van Der Aalst, W.: Data science in action. In: *Process mining: Data science in action*, pp. 3–23. Springer (2016)
24. Ward Jr, J.H.: Hierarchical grouping to optimize an objective function. *Journal of the American statistical association* pp. 236–244 (1963)
25. Weinzierl, S.: Exploring gated graph sequence neural networks for predicting next process activities. In: *BPM*. pp. 30–42. Springer (2021)
26. Weytjens, H., De Weerd, J.: Creating unbiased public benchmark datasets with data leakage prevention for predictive process monitoring. In: *BPM*. pp. 18–29. Springer (2021)
27. Wuyts, B., Vanden Broucke, S., De Weerd, J.: Sutran: an encoder-decoder transformer for full-context-aware suffix prediction of business processes. In: *ICPM*. pp. 17–24. IEEE (2024)
28. Zuo, S., Jiang, H., Li, Z., Zhao, T., Zha, H.: Transformer hawkes process. In: *ICML*. pp. 11692–11702. PMLR (2020)